

End-to-End Machine Learning Workflow & Deployment

Official Project Documentation — Diabetes Prediction Web Application

1. Executive Summary & Architecture Overview

This technical documentation outlines the end-to-end framework implemented to train, serialize, and deploy a predictive Machine Learning model. The project leverages the **PIMA Diabetes Dataset** to construct a binary classification system that assesses whether an individual is diabetic based on clinical measurements.

The lifecycle of this project spans three core environments:

1. **Jupyter Notebook:** Utilized for exploratory data analysis, preprocessing, training, and model selection.
2. **Spyder IDE:** Used to design structural python modules, test local production code, and formulate the user interface.
3. **Streamlit Application Framework:** Deployed as a web app to provide end-users with real-time reactive inferences.

Architecture Pipeline: Raw Data (.csv) → Data Scaling & Partitioning → SVM Model Training → Serialization (.sav) → Spyder Local Dev → Streamlit Live Server

2. Phase 1: Model Training & Evaluation (Jupyter Notebook)

The development begins within a Jupyter Notebook environment where the dependencies are imported, the dataset is cleaned, and a Support Vector Machine (SVM) classifier with a linear kernel is trained.

Step 2.1: Importing Dependencies

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.metrics import accuracy_score
import pickle
```

Step 2.2: Data Collection and Analysis

The dataset contains features like `Pregnancies`, `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, `BMI`, `DiabetesPedigreeFunction`, and `Age`. The target column is `Outcome` where:

- `Outcome = 1` indicates a diabetic diagnosis.
- `Outcome = 0` indicates a non-diabetic diagnosis.

```
# Loading the diabetes dataset
df = pd.read_csv('diabetes.csv')

# Splitting features and target variables
X = df.drop(columns='Outcome', axis=1)
Y = df['Outcome']
```

Step 2.3: Data Partitioning and Model Optimization

The dataset is split into training and test sets using a stratified approach to keep balanced outcome distributions. An SVM model is then fitted to the training set.

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, stratify=Y,
random_state=2)

classifier = svm.SVC(kernel='linear')
classifier.fit(X_train, Y_train)
```

Step 2.4: Model Serialization (Saving the Trained Model)

To transition the model from training to deployment, the object state must be serialized using the `pickle` library and exported as a `.sav` or `.pkl` file.

```
filename = 'trained_model.sav'
pickle.dump(classifier, open(filename, 'wb'))
```

3. Phase 2: Production Testing & Verification (Spyder)

Once the file `trained_model.sav` is exported, we migrate to the **Spyder IDE** to build a standalone predictive system and verify that the model safely unpacks and behaves as expected outside of interactive notebook blocks.

Step 3.1: Python Predictive Script (`prediction_system.py`)

Create a clean Python file to handle model verification. The script deserializes the file and conducts inference on a mock patient record matrix.

```

import numpy as np
import pickle

# Loading the serialized model object
loaded_model = pickle.load(open('trained_model.sav', 'rb'))

# Mock instance definition
input_data = (5, 166, 72, 19, 175, 25.8, 0.587, 51)
input_data_as_numpy_array = np.asarray(input_data)
input_data_reshaped = input_data_as_numpy_array.reshape(1, -1)

# Inference execution
prediction = loaded_model.predict(input_data_reshaped)

if (prediction[0] == 1):
    print('The person is diabetic')
else:
    print('The person is not diabetic')

```

4. Phase 3: Building & Deploying the Web App (Streamlit)

The final stage maps user interaction components directly onto the inference engine using Streamlit. The interface handles clinical metric inputs and serves real-time predictive diagnostic flags.

Step 4.1: Constructing the Application Script (**Diabetic Prediction Web App.py**)

Develop the production application code structure inside your Spyder IDE:

```

import numpy as np
import pickle
import streamlit as st

# Safe model load
loaded_model = pickle.load(open('trained_model.sav', 'rb'))

def diabetes_prediction(input_data):
    input_data_as_numpy_array = np.asarray(input_data, dtype=float)
    input_data_reshaped = input_data_as_numpy_array.reshape(1, -1)
    prediction = loaded_model.predict(input_data_reshaped)

    if (prediction[0] == 0):
        return 'The person is not diabetic'
    else:
        return 'The person is diabetic'

def main():
    st.title('Diabetes Prediction Web Application')

```

```

# Input field assignments
Pregnancies = st.text_input('Number of Pregnancies')
Glucose = st.text_input('Glucose Level (mg/dL)')
BloodPressure = st.text_input('Blood Pressure Value (mm Hg)')
SkinThickness = st.text_input('Skin Thickness Value (mm)')
Insulin = st.text_input('Insulin Level (mu U/ml)')
BMI = st.text_input('BMI Value (Weight in kg/(Height in m)^2)')
DiabetesPedigreeFunction = st.text_input('Diabetes Pedigree Function Value')
Age = st.text_input('Age of the Person')

diagnosis = ''

if st.button('Run Diabetes Test Result'):
    # Pass captured strings inside a tuple block
    diagnosis = diabetes_prediction([
        Pregnancies, Glucose, BloodPressure, SkinThickness,
        Insulin, BMI, DiabetesPedigreeFunction, Age
    ])

    st.success(diagnosis)

if __name__ == '__main__':
    main()

```

Step 4.2: Running the Web App Deployment Locally

Streamlit files cannot be run standardly through individual IDE script executions. Instead, they require a dedicated runtime environment initialized via terminal execution:

1. Open your system Terminal, Command Prompt, or the integrated terminal inside Spyder.
2. Navigate to the folder directory containing your python web app script:

```
cd "C:/Users/YourPath/ProjectFiles"
```

3. Launch the application server local thread using the execution framework below:

```
streamlit run "Diabetic Prediction Web App.py"
```

4. The terminal will output a local network address (typically `http://localhost:8501`) and automatically forward the rendering layer directly to your default internet browser window.

5. Deployment Checklist & Troubleshooting

- **Path Management:** Ensure that `trained_model.sav` sits in the exact active directory path as your running web application script, or replace it with an absolute file system path.

- **Package Requirements:** Verify production dependencies match exact versions across your environments. Your base dependencies must include: `scikit-learn`, `streamlit`, `numpy`, and `pandas`.
- **Data Structuring:** Ensure inputs passed via text fields are parsed correctly as floats or integers before entering the `numpy.asarray()` conversion pipeline to prevent structural matrix validation errors.